Class 5: Data Viz with ggplot2

Mayla (PID: 19136116)

Table of contents

Background

There are many graphics systems in R for making plots and figures. These include so-called *"base R" graphics* like the `plot()` function and add on packages like **ggplot2**.

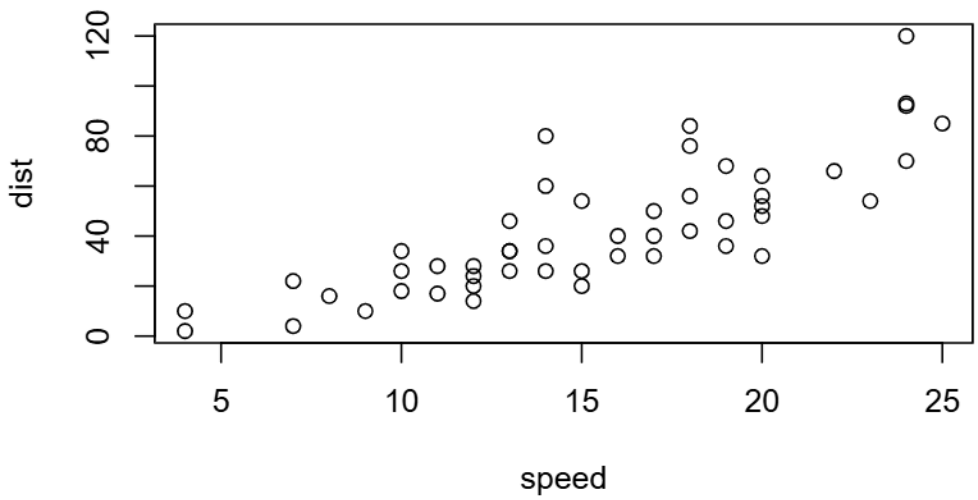
Let's compare how we make a simple figure with these two systems:

We can use the in-built `cars` dataset:

```
head( cars )
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

```
plot(cars)
```



Before I can use ggplot2 I need to instsall it on my computer. To do this we can use the function `install.packages("ggplot2")`

> **N.B.** We never run `install.packages()` in our quarto doc (we run it once only in our R console) as it would re-install the package every time we render our quarto report.

Once installed we need to load up the package into our R brain:

```
library(ggplot2)
```

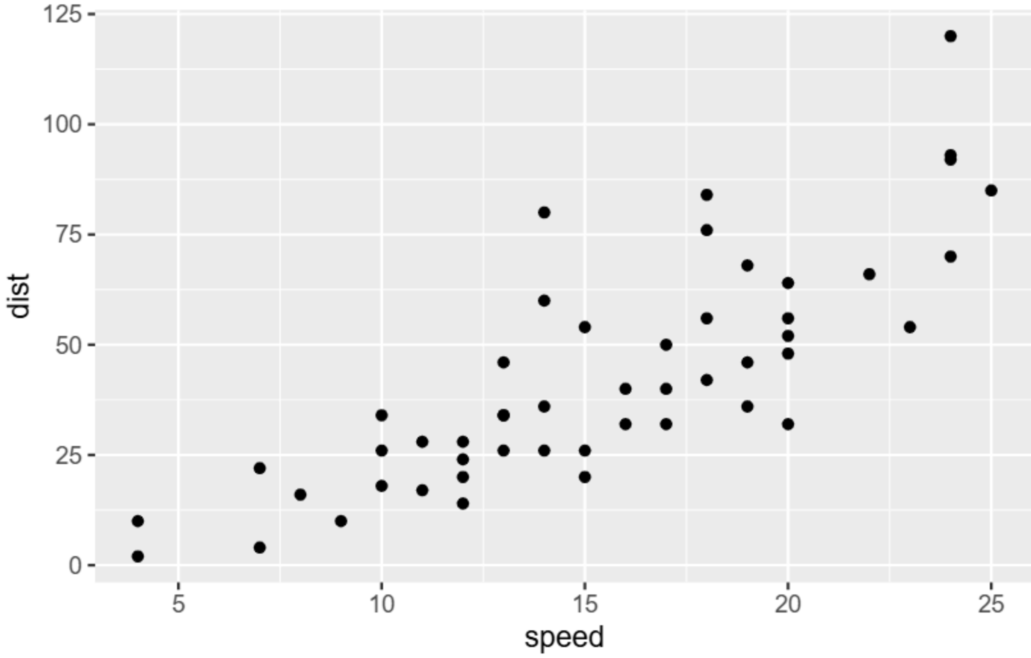
The main function in the **gplot2** package is called `ggplot()`

```
ggplot(cars)
```

Every ggplot has at least 3 layers:

- the **data** (a data.frame of the stuff we want to plot)
- the **aesthetics** (how the data maps to the plot),
- the **geom** layer (how you want the plot drawn, e.g. points, lines, etc.)

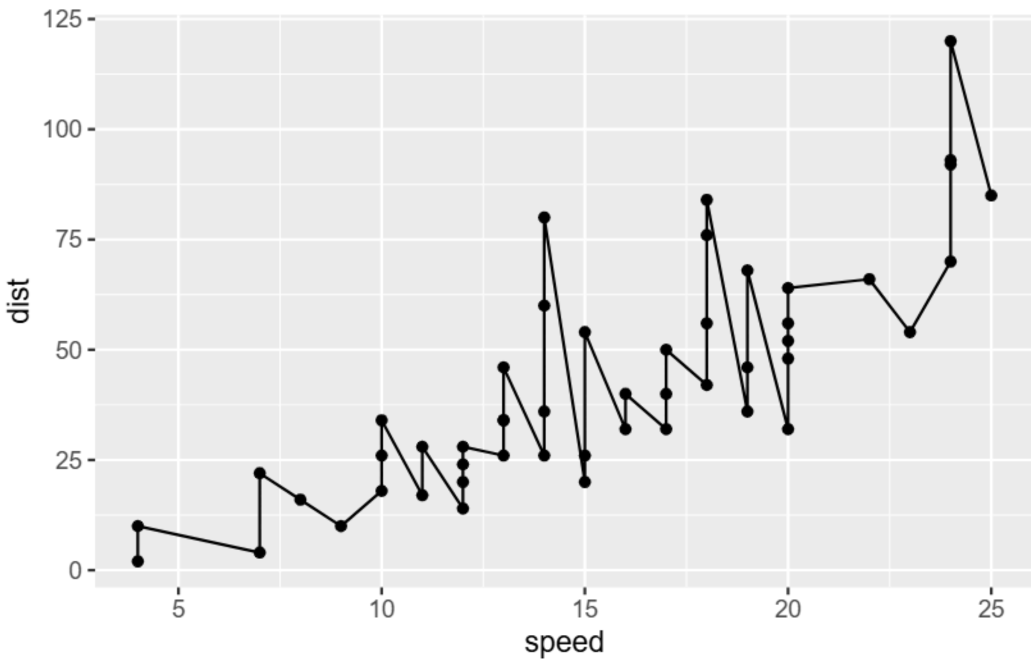
```
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point()
```



Add some custom features

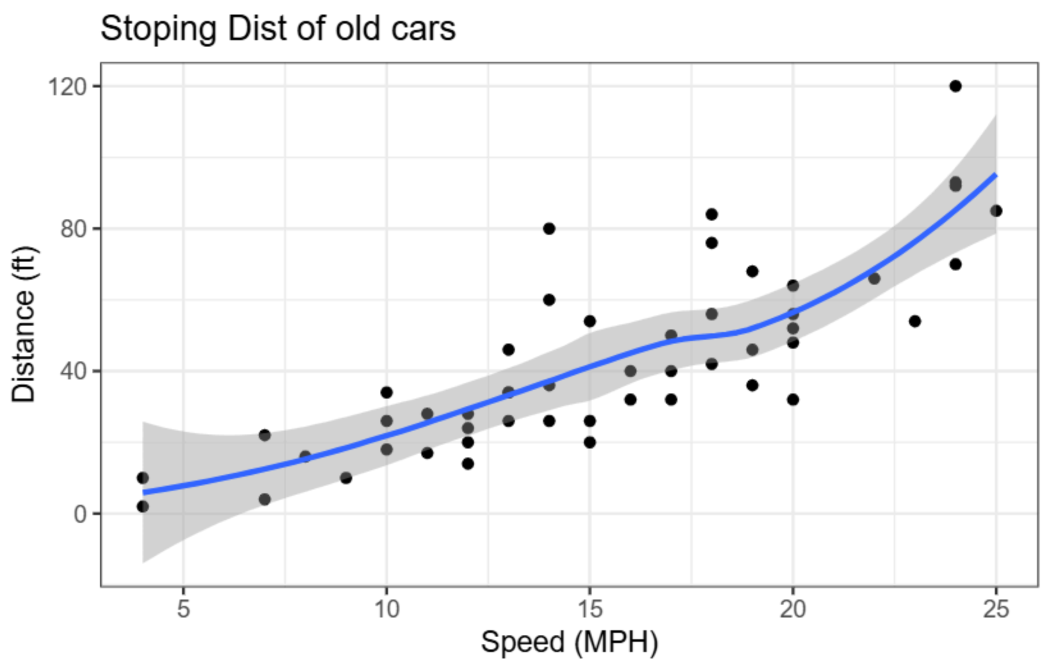
Let's add a trend line that shows the relationship between speed and distance.

```
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_line()
```




```r
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_smooth() +
  theme_bw() +
  labs(title = "Stoping Dist of old cars",
       x = "Speed (MPH)",
       y = "Distance (ft)")
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

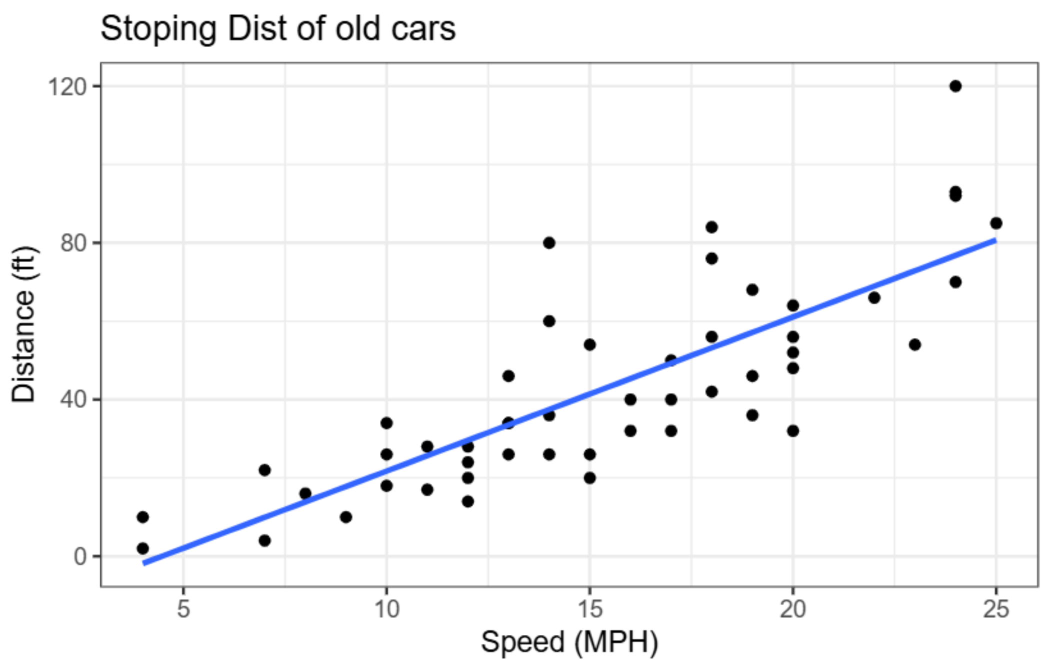


Q. Can you make the `geom_smooth()` function produce a linear straight line fit to the data and turn off the "gray" error region.

Gene Expression Figure

```
ggplot(cars) +
  aes(x = speed, y = dist) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  theme_bw() +
  labs(title = "Stoping Dist of old cars",
       x = "Speed (MPH)",
       y = "Distance (ft)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
genes <- read.delim(url)
head(genes)
```

```
        Gene Condition1 Condition2      State
1      A4GNT -3.6808610 -3.4401355 unchanging
2       AAAS  4.5479580  4.3864126 unchanging
3      AASDH  3.7190695  3.4787276 unchanging
4       AATF  5.0784720  5.0151916 unchanging
5       AATK  0.4711421  0.5598642 unchanging
6 AB015752.4 -3.6808610 -3.5921390 unchanging
```

```
sum(genes$State == "up")
```

```
[1] 127
```

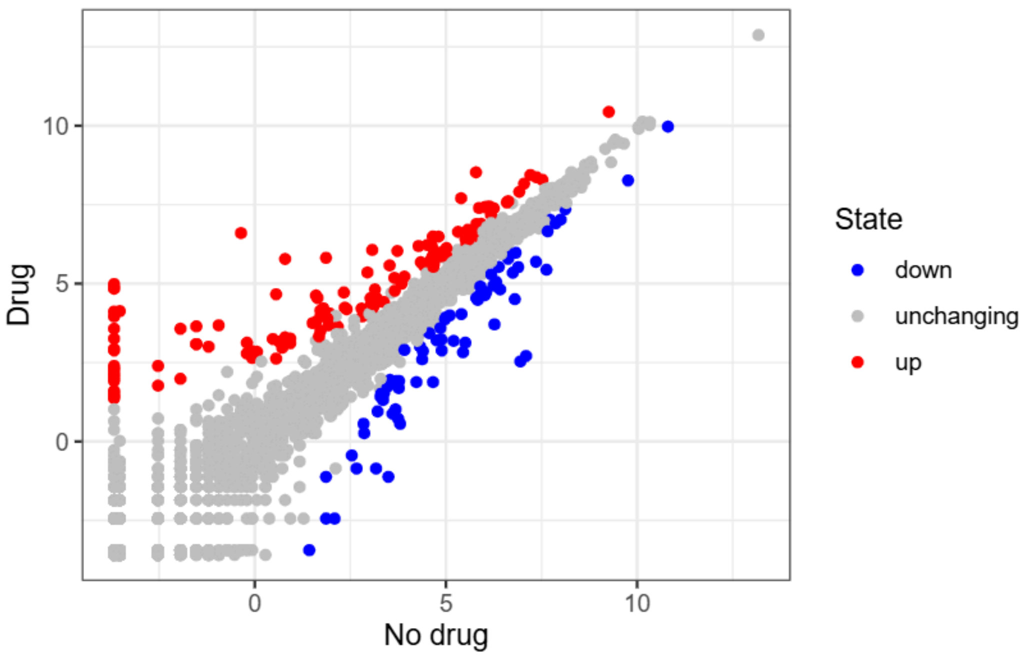
A useful new function in this context is the `table()` function:

```
table(genes$State)
```

```
     down unchanging         up 
       72       4997        127 
```

My first plot attempt

```
ggplot(genes) +
  aes(Condition1, Condition2, col=State) +
  geom_point() +
scale_color_manual(values = c("blue","gray","red")) +
theme_bw() +
  labs(x= "No drug", y = "Drug")
```



Going further

Here we read the famous gapminder dataset.

```
# File location online
url <- "https://raw.githubusercontent.com/jennybc/gapminder/master/inst/extdata/gapminder.tsv

gapminder <- read.delim(url)
head(gapminder)
```

```
      country continent year lifeExp      pop gdpPercap
1 Afghanistan      Asia 1952  28.801  8425333  779.4453
2 Afghanistan      Asia 1957  30.332  9240934  820.8530
3 Afghanistan      Asia 1962  31.997 10267083  853.1007
4 Afghanistan      Asia 1967  34.020 11537966  836.1971
5 Afghanistan      Asia 1972  36.088 13079460  739.9811
6 Afghanistan      Asia 1977  38.438 14880372  786.1134
```

Q. How many entries (i.e. rows) are in this dataset?

```r
nrow(gapminder)
```

```
[1] 1704
```

Q. How many different "country" entries are in this dataset?

```r
length (table(gapminder$country))
```

```
[1] 142
```

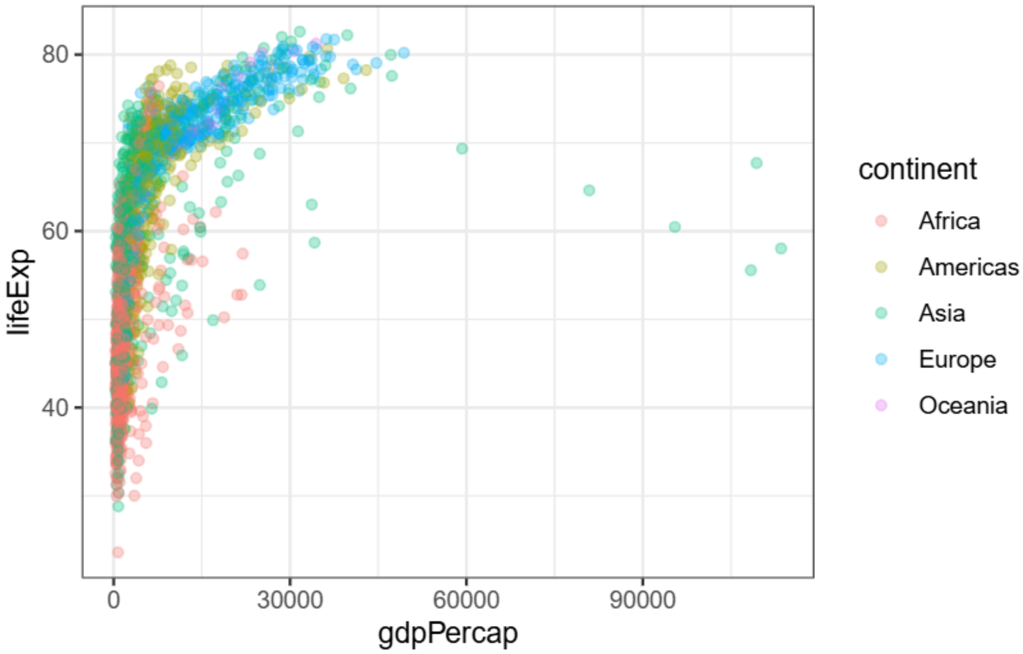
```r
length (unique (gapminder$country))
```

```
[1] 142
```

Let's make our first plot of the entire dataset:

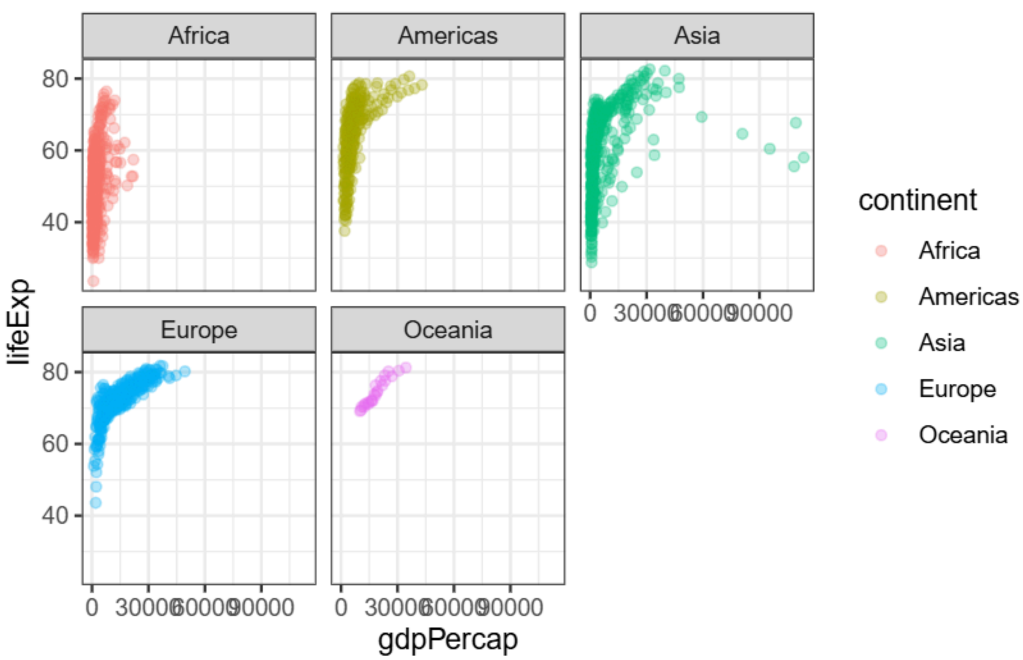
```r
p <- ggplot(gapminder) +
  aes(x = gdpPercap, y = lifeExp, col = continent) +
  geom_point(alpha=0.3) +
theme_bw()
```

```r
p
```



I can add more layers to `p`

```r
p +
  facet_wrap(~continent)
```



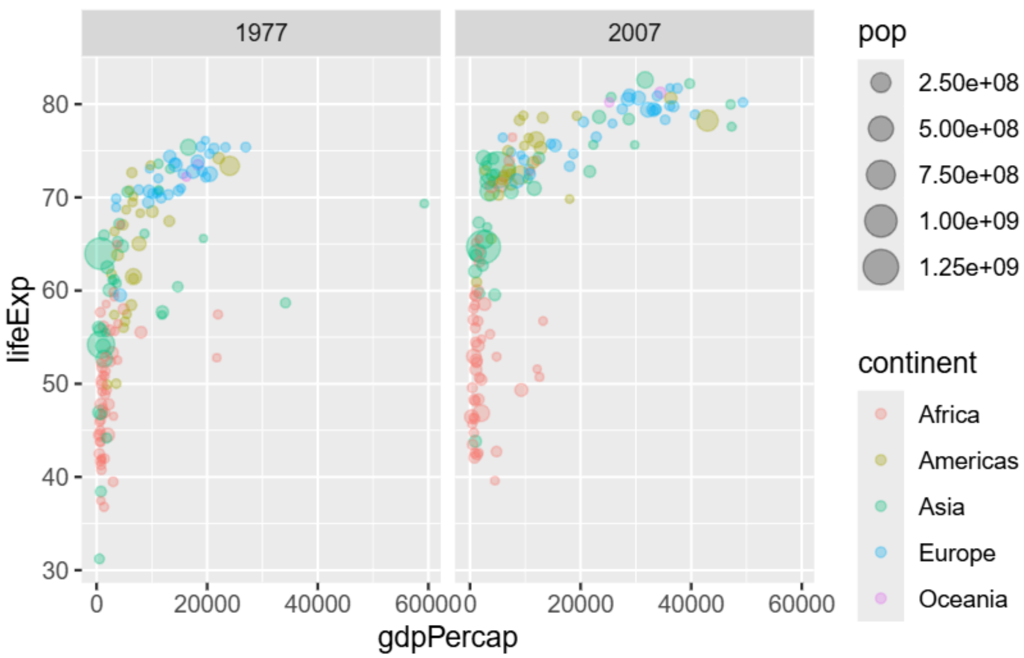
Make a plot for years 1974 and 1978 only (not all the years in the dataset).

Q. First use the **dplyr** package and the `filter()` function from that package to extract the rows from the year 1978.

```
library(dplyr)
```

```
g07 <- filter(gapminder, year == 2007)
g77 <- filter(gapminder, year==1977)
g <- filter(gapminder, year == 2007 | year == 1977)
```

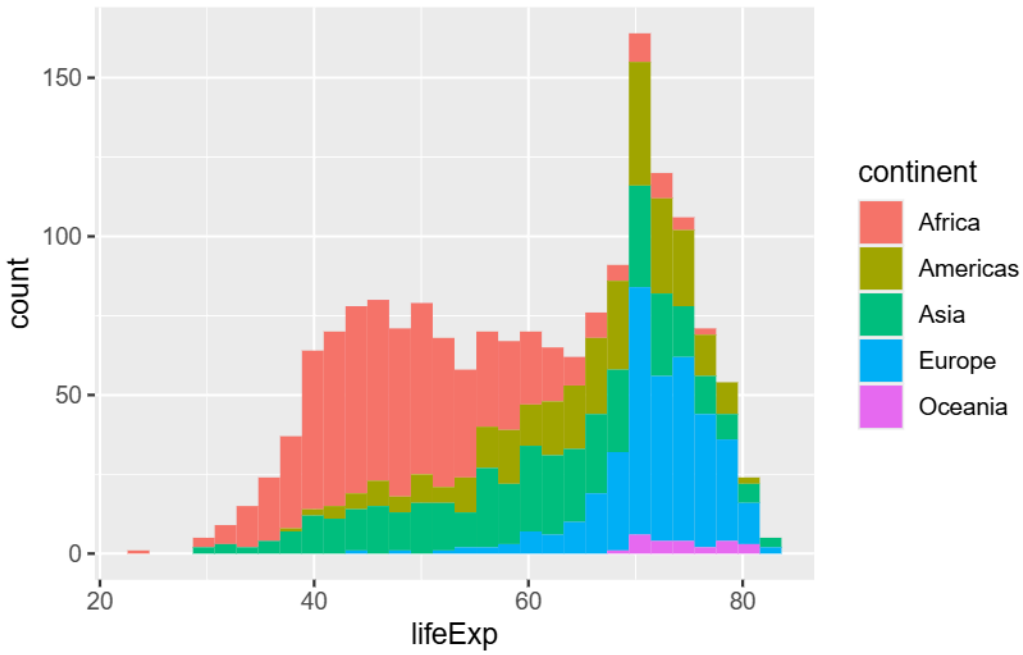
```
ggplot(g) +
  aes(x = gdpPercap, y = lifeExp, col = continent, size = pop) +
  geom_point(alpha=0.3) +
facet_wrap(~year)
```



Q. Make a histogram of lifeExp colored by continent (try using `fill=continent` or `col=continent`)

```
ggplot(gapminder) +
  aes(lifeExp) +
  geom_histogram(aes(fill=continent))
```

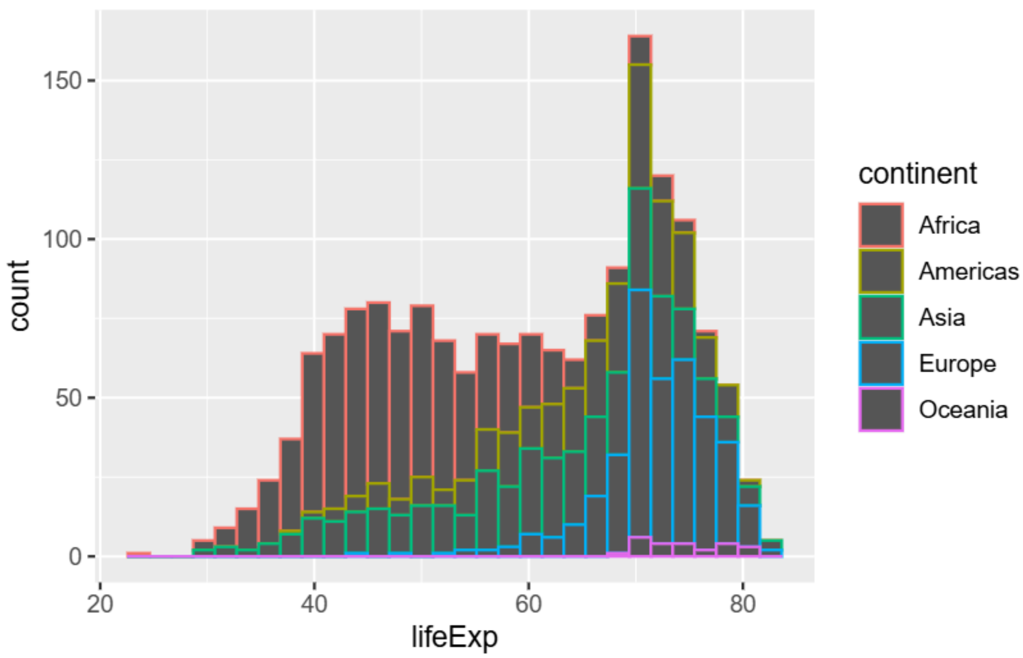
```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



OR

```r
ggplot(gapminder) +
  aes(lifeExp) +
  geom_histogram(aes(col=continent))
```

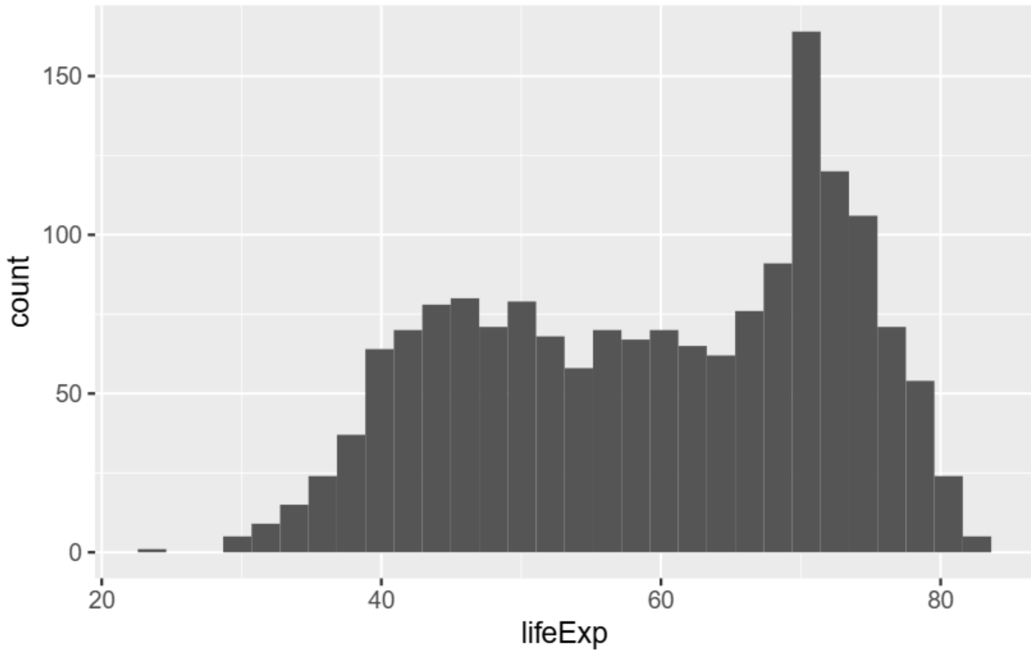
```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Q. Make a histogram of lifeExp faceted by continent

```
ggplot(gapminder) +
  aes(lifeExp) +
  geom_histogram()
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```r
facet_wrap(~continent)
```

```
<ggproto object: Class FacetWrap, Facet, gg>
    attach_axes: function
    attach_strips: function
    compute_layout: function
    draw_back: function
    draw_front: function
    draw_labels: function
    draw_panel_content: function
    draw_panels: function
    finish_data: function
    format_strip_labels: function
    init_gtable: function
    init_scales: function
    map_data: function
    params: list
    set_panel_size: function
    setup_data: function
    setup_panel_params: function
    setup_params: function
    shrink: TRUE
```

```
train_scales: function
vars: function
super:  <ggproto object: Class FacetWrap, Facet, gg>
```